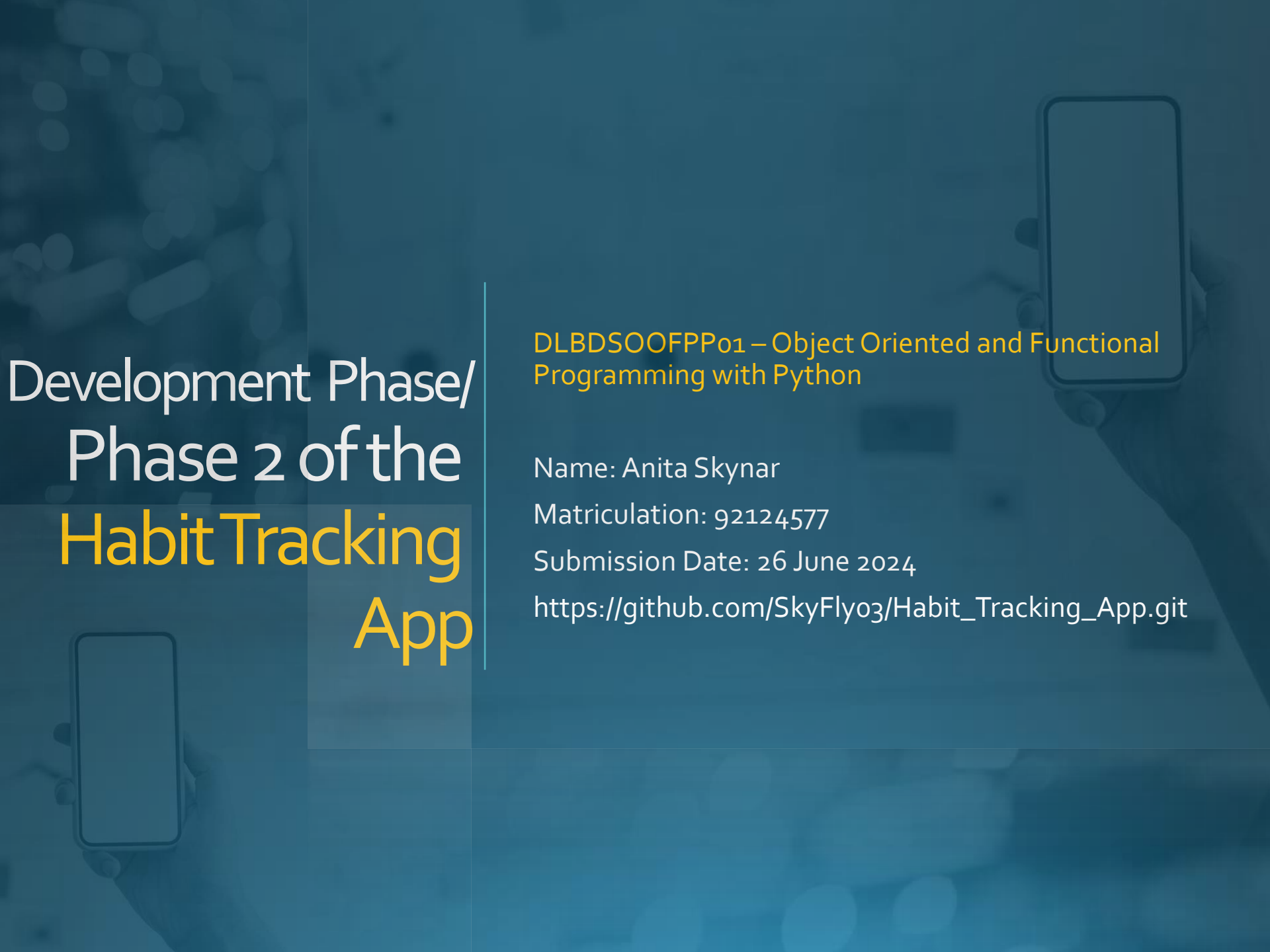




Development Phase/ Phase 2 of the Habit Tracking App

DLBDSOOFPP01 – Object Oriented and Functional
Programming with Python

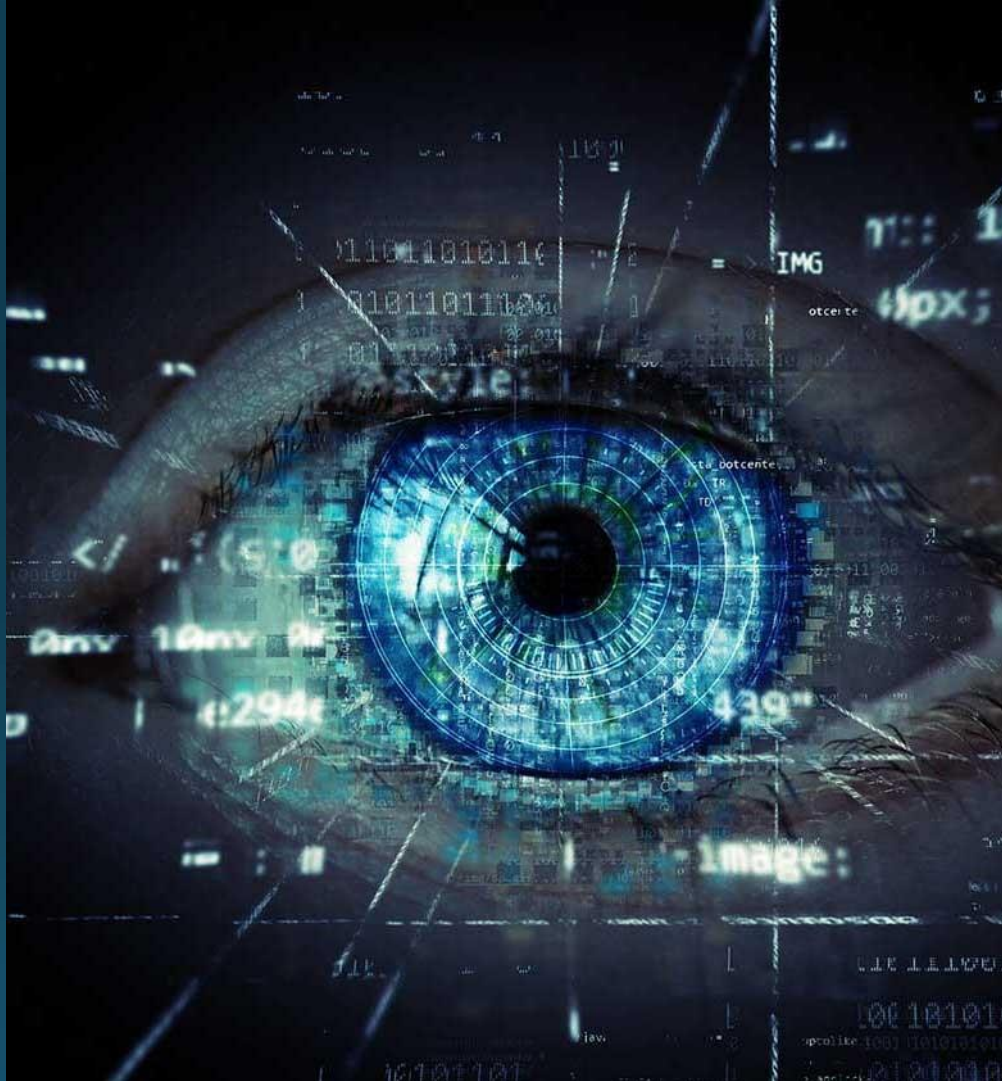
Name: Anita Skynar

Matriculation: 92124577

Submission Date: 26 June 2024

https://github.com/SkyFly03/Habit_Tracking_App.git

Technical Set Up



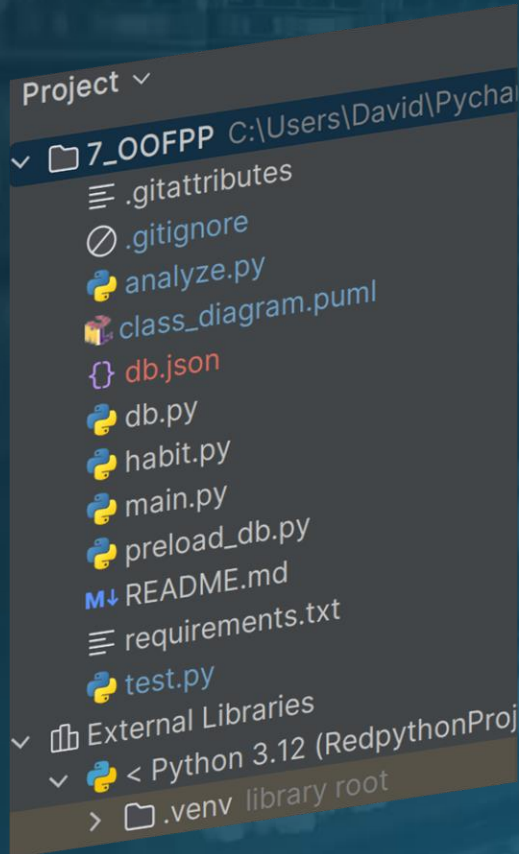
- **Tools Used:** PyCharm IDE
- **Python 3.7** or later
- **Virtual Environment** setup

```
python -m venv .venv  
.venv\Scripts\activate
```
- **Required Libraries:**
questionary, pytest, JSON
- **Install Dependencies:**

```
pip install -r requirements.txt
```
- **Executing Scripts in the Terminal:**

```
python preload_db.py  
python main.py
```

Project Directory Structure



User Interaction (main.py)

```
~~~~~  
Welcome! Ready to build some healthy habits? Select an option from the menu below:  
~~~~~  
(Use arrow keys)
```

```
» Add Habit  
View Habits  
Analyze Habit  
Log Entry  
Delete Habit  
Exit
```

```
def main():  
    log_datetime = datetime.strptime(__date_string: f"{date_str} {time_str}", __format: '%Y-%m-%d  
else:  
    log_datetime = datetime.now().replace(second=0, microsecond=0)  
  
    habit.log_entry(log_datetime)  
    db.save_db()  
    print(f"Logged entry for habit: {habit.name} on {log_datetime.strftime('%Y-%m-%d %H:%M')}")  
except ValueError:  
    print("Invalid date or time format. Please use YYYY-MM-DD for date and HH:MM for time.")  
    print("Returning to the main menu.")
```

Description:

CLI for user interaction using questionnaire library

Features :

Adding, viewing and analyzing habits, logging entries and deleting habits, exit

Error Handling:

Returns to main menu on input mistakes

Habit Module (habit.py)

Importance:

Organizes and
reuses related data
and functions

Attributes:

name, start_date,
log, periodicity

Methods:

__init__()
calculate_streak()
log_entry()

```
class Habit:
    def calculate_streak(self):
        streak = 0
        today = datetime.today().date()

        if not self.log:
            return streak

        sorted_log = sorted(self.log, reverse=True)
        current_date = sorted_log[0].date()

        if self.periodicity == 'daily':
            for log_datetime in sorted_log:
                log_date = log_datetime.date()
                if log_date == current_date:
                    streak += 1
                    current_date -= timedelta(days=1)
                else:
                    break
        elif self.periodicity == 'weekly':
            for log_datetime in sorted_log:
                log_date = log_datetime.date()
```


Data Storage & Retrieval (db.py)

```
class Database:
    def load_db(self):
        try:
            with open(self.db_file, "r") as file:
                data = json.load(file)
                habits = []
                for item in data:
                    habits.append(Habit(
                        name=item['name'],
                        start_date=datetime.strptime(item['start_date'], '%Y-%m-%d %H:%M'),
                        log=[datetime.strptime(log_datetime, '%Y-%m-%d %H:%M') for log_datetime in item['log']],
                        periodicity=item.get('periodicity', 'daily')
                    ))
                return habits
        except FileNotFoundError:
            return []

    def save_db(self):
```

Description:
Manages habit data storage and retrieval

Purpose:
Ensures data persistence between user sessions

Key functions:

- load_db()
- save_db()
- get_all_habits()
- get_habit()
- add_habit()
- delete_habit()
- get_habits_by_periodicity()
- get_longest_streak_all_habits()
- get_longest_streak_for_habit()

JSON Data Format (db.json)

Description:
Stores habit data in JSON format

Structure:
List of habit dictionaries

Each habit contains:
name
start_date
log
and periodicity

```
{
  "name": "Calling Grandma",
  "start_date": "2024-05-17 17:32",
  "log": [
    "2024-05-17 17:32",
    "2024-05-25 08:00",
    "2024-06-01 09:00",
    "2024-06-08 10:00",
    "2024-06-15 11:00",
    "2024-06-22 12:00",
    "2024-06-29 13:00"
  ],
  "periodicity": "weekly"
}
```

Analytics Module (analyze.py)

Description:

Provides functionalities for habit analysis

Function:

analyze_habits()

User Interaction:

Uses the questionnaire library
Asking users to choose a habit from the database

Explanation:

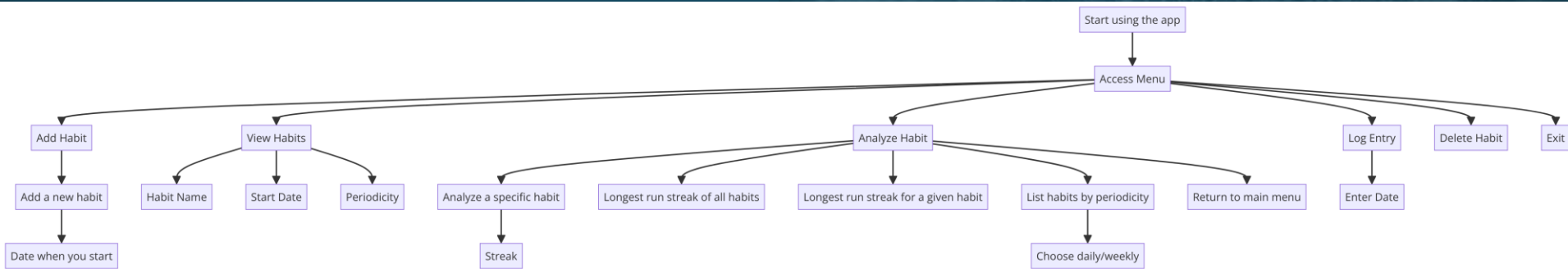
Gives users insights into their habit performance and consistency

```
Analyze Habit
? Select an analytical function: (Use arrow keys)
» Analyze a specific habit
   Longest run streak of all habits
   Longest run streak for a given habit
   List habits by periodicity
   Return to main menu
```

```
Analyze Habit
? Select an analytical function: Analyze a specific habit
? Select a habit to analyze: (Use arrow keys)
» Stretching
   No Netflix Binging
   Learning Japanese
   Calling Grandma
   Cleaning Apartment
   stop chewing gum
```

```
Analyze Habit
Select an analytical function: List habits by periodicity
Select the periodicity: (Use arrow keys)
» daily
   weekly
```

Flow Chart Diagram



1

2

3

4

5

6

Add Habits:

With start date, and periodicity

View All Habits:

Shown by name, start date, and periodicity

Analyze Habits:

A specific habit can be analyzed, the longest run streak for all habits, for a given habit, list habits by periodicity

Log Habits:

Enables the user to log a completion date and time for a habit

Delete Habits:

Permits users to remove habits they no longer want to track

Exit:

Users can exit the application at any time

UML Class Diagram

Shows relationships and dependencies between the 5 classes in the habit tracking app

(test.py)

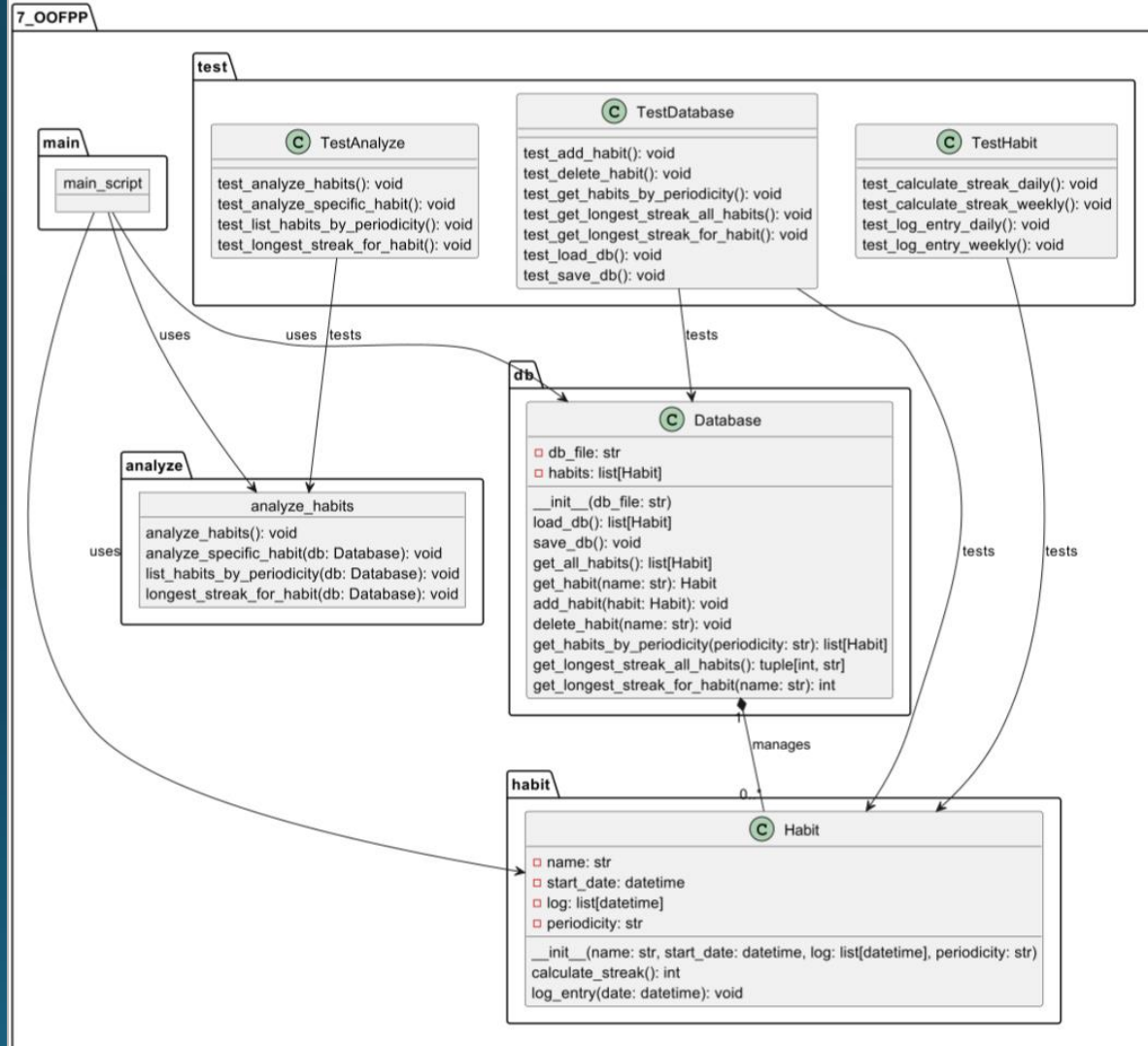
TestAnalyze Class: Tests methods of the Analyze class

TestDatabase Class: Tests methods of the Database class

TestHabit Class: Tests methods of the Habit class

(db.py) **Database Class:**
Handles storage and retrieval of habits using JSON, including methods to load/save, add/delete habits, and calculate streaks

(habit.py) **Habit Class:**
Manages individual habit details and behaviors, including methods to log entries and calculate streaks



Unit Testing (test.py)

Purpose:

Ensures code reliability

```
python test.py
```

Documentation (README.md)

README.md: Instructions on installation, usage, testing

Format: Markdown, compatible with GitHub

```
===== test session starts =====
collecting ... collected 15 items

test.py::TestHabit::test_calculate_streak_daily PASSED [ 6%]
test.py::TestHabit::test_calculate_streak_weekly PASSED [ 13%]
test.py::TestHabit::test_log_entry_daily PASSED [ 20%]
test.py::TestHabit::test_log_entry_weekly PASSED [ 26%]
test.py::TestDatabase::test_add_habit PASSED [ 33%]
test.py::TestDatabase::test_delete_habit PASSED [ 40%]
test.py::TestDatabase::test_get_habits_by_periodicity PASSED [ 46%]
test.py::TestDatabase::test_get_longest_streak_all_habits PASSED [ 53%]
test.py::TestDatabase::test_get_longest_streak_for_habit PASSED [ 60%]
test.py::TestDatabase::test_load_db PASSED [ 66%]
test.py::TestDatabase::test_save_db PASSED [ 73%]
test.py::TestAnalyze::test_analyze_habits PASSED [ 80%]
test.py::TestAnalyze::test_analyze_specific_habit PASSED [ 86%]
test.py::TestAnalyze::test_list_habits_by_periodicity PASSED [ 93%]
test.py::TestAnalyze::test_longest_streak_for_habit PASSED [100%]

===== 15 passed in 0.81s =====
Process finished with exit code 0
```